
Génie Logiciel

CER | Synchronisation

Lundi 16 février 2026

TABLE DES MATIÈRES

Étude de cas : Burnout	2
AAV	2
Mots clés	3
Contexte	3
Problématique	3
Contraintes	3
Généralisation	3
Livrables	3
Pistes de solutions	4
Plan d'actions	4
Démarche : Burnout	5
Définition: Mots clés	5
Corbeille check	12
Workshop	16
Analyse et résolution : Burnout	17
Conclusion	17
Capitalisation	18
Ressources	19

ÉTUDE DE CAS : BURNOUT

AAV

- 1 Définir Thread, Multi-threading, Mécanismes d'IPC, Ressource critique / Section critique, inter-blocage
- 1 Citer, Décrire et expliquer les principaux problèmes liés à la section critique
- 1 Citer, Décrire et expliquer les principaux mécanismes d'IPC et les organiser en catégories
- 3 Implémenter les mécanismes de synchronisation en C#

MOTS CLÉS

- Synchronisation
- Cœurs logiques
- Séquentiellement
- Multi serveur

CONTEXTE

Le client du big data utilise une chaîne de traitement de données réparti sur 3 serveurs avec des processeurs différents. Certaines données ne sont jamais traitées et l'intégrité n'est pas garantie.

PROBLÉMATIQUE

Comment garantir l'intégrité des données en mettant en place une synchronisation inter-serveur?

CONTRAINTES

- Architecture des serveurs
- Délai : 2 jours
- Intégrité des données

GÉNÉRALISATION

- Multiserveurs
- Synchronisation

LIVRABLES

- Plan de synchronisation
- Diagrammes et code
- Rapport

PISTES DE SOLUTIONS

- Étudier les différents types de synchronisation
- Multitasking

PLAN D' ACTIONS

- Workshops
- Diagrammes
- Faire le code

DÉMARCHE : BURNOUT

DÉFINITION : MOTS CLÉS

Synchronisation: La synchronisation se présente comme un processus qui permet de faire correspondre, c'est-à-dire conserver à l'identique, les contenus de deux ou plusieurs emplacements de stockage.

Synchronisation unidirectionnelle et bidirectionnelle:

En informatique, la synchronisation peut être unidirectionnelle ou bidirectionnelle. La différence réside dans le sens où la duplication des données se fait.

Lors d'une **synchronisation unidirectionnelle**, la duplication des mises à jour effectuées se fait dans un sens unique.

C'est-à-dire les modifications réalisées sur l'endroit A sont transcrites à l'endroit B mais celles effectuées à l'endroit B n'affectent pas les données enregistrées à l'endroit A.

Par contre, lors d'une **synchronisation bidirectionnelle**, la transcription des modifications se fait dans les deux sens.

C'est-à-dire peu importe l'endroit où la modification est effectuée, elle sera toujours reproduite vers l'autre endroit.

Cœurs logiques: Les « threads » ou « cœurs logiques » (par opposition aux cœurs physiques) sont une file d'attente de commandes qui permet d'effectuer des opérations sur les cœurs physiques. Vous aurez au moins un thread par cœur, même si dans les processeurs modernes, il est désormais courant d'en avoir deux.

Un **cœur de processeur** comporte plusieurs zones de calcul différentes, telles que l'ALU et le FPU, qui sont techniquement indépendantes mais font toujours partie de ce cœur. Les instructions (pensez au calcul) sont exécutées de manière linéaire car une instruction future peut dépendre du résultat d'une instruction passée. Cela signifie que l'ALU peut être pleinement utilisée et le FPU inutilisé en attendant les résultats. Ici, nous ajoutons une deuxième file d'attente de commandes via le noyau qui peut utiliser ces zones de calcul inutilisées pour effectuer davantage de travail. Autrement dit, nous disposons désormais de deux « threads » d'instructions via un seul noyau physique.

Du côté logiciel, nous avons un autre concept similaire également appelé « threads », qui consiste simplement en plusieurs processus distincts fonctionnant sous un seul processus. Le processus principal gère les données entre les threads. Chaque thread ne peut être exécuté que sous un seul cœur ou un seul cœur logique, donc un processus créant de nombreux threads logiciels est un moyen pour lui de fonctionner simultanément sur plusieurs cœurs logiques.

Séquentiellement: De manière séquentielle signifie que les tâches ou les opérations sont exécutées les unes après les autres, dans un ordre spécifique.

Multi serveur: Le multiserveur, terme informatique, désigne une configuration qui implique l'utilisation de plusieurs serveurs d'information pour offrir des services et gérer les données. Cette solution est généralement adoptée pour améliorer la performance, la sécurité et la disponibilité des ressources en ligne. Dans un système multiserveur, chaque serveur peut avoir un rôle spécifique ou être réparti en fonction des besoins de l'application.

Avantages du multiserveur :

Cette approche permet de réduire la charge sur un seul serveur, optimisant ainsi les performances globales du système et assurant une meilleure répartition des tâches entre les différents serveurs. En outre, le multiserveur offre une plus grande résilience face aux pannes matérielles ou logicielles, car il est possible de basculer rapidement vers un autre serveur en cas de problème.

Interblocage:

Un interblocage, ou deadlock en anglais, est une situation dans laquelle deux ou plusieurs processus ou threads sont bloqués de manière permanente, chacun attendant que l'autre libère une ressource dont il a besoin pour continuer son exécution.

Comment résoudre ?

Pour résoudre un interblocage, il existe plusieurs approches, notamment :

- **Prévention** : Concevoir le système de manière à éviter les conditions d'interblocage, par exemple en imposant un ordre strict d'acquisition des ressources.
- **Détection** : Mettre en place des mécanismes pour détecter les interblocages lorsqu'ils se produisent, puis prendre des mesures pour les résoudre, comme tuer un processus ou libérer des ressources.
- **Évitement** : Utiliser des algorithmes d'évitement d'interblocage, tels que l'algorithme du banquier, qui permet de déterminer si une demande de ressource peut être satisfaite sans entraîner un interblocage.
- **Récupération** : Si un interblocage est détecté, prendre des mesures pour récupérer le système, comme redémarrer les processus impliqués ou libérer manuellement les ressources.
- **Utilisation de timeout** : Imposer des délais d'attente pour les processus qui attendent des ressources, afin de détecter et de résoudre les interblocages plus rapidement.

Famine:

La famine, dans le contexte de l'informatique, fait référence à une situation où un processus ou un thread ne parvient pas à accéder à une ressource partagée pendant une période prolongée, en raison de la concurrence avec d'autres processus ou threads.

Comment résoudre ?

Pour résoudre la famine, plusieurs stratégies peuvent être mises en place, notamment :

- **Priorisation équitable** : Mettre en place un système de priorisation qui garantit que tous les processus ou threads ont une chance équitable d'accéder aux ressources partagées, en évitant que certains ne soient constamment favorisés au détriment des autres.
- **Rotation des priorités** : Utiliser une approche de rotation des priorités, où les processus ou threads qui ont été ignorés pendant un certain temps voient leur priorité augmenter temporairement pour leur permettre d'accéder aux ressources.
- **Limitation du temps d'attente** : Imposer des limites de temps d'attente pour les processus ou threads qui attendent des ressources, afin de garantir qu'ils ne restent pas bloqués indéfiniment.
- **Utilisation de sémaphores ou de mutex** : Mettre en place des mécanismes de synchronisation tels que des sémaphores ou des mutex pour contrôler l'accès aux ressources partagées et éviter que certains processus ou threads ne soient constamment bloqués.
- **Surveillance et ajustement** : Mettre en place des outils de surveillance pour détecter les situations de famine et ajuster les politiques de gestion des ressources en conséquence, afin d'assurer une répartition plus équitable.

Section critique:

Une section critique est une partie d'un programme où l'accès à une ressource partagée doit être contrôlé pour éviter les conflits entre plusieurs processus ou threads. C'est comme une salle de réunion où seules certaines personnes peuvent entrer à la fois pour discuter d'un sujet important, afin d'éviter les interruptions et les malentendus.

Flag:

Un flag, ou drapeau en français, est une variable utilisée pour indiquer l'état ou la condition d'un processus ou d'une opération. C'est comme un signal lumineux qui peut être allumé ou éteint pour indiquer si une certaine condition est vraie ou fausse.

Sémaphore:

Un sémaphore est un mécanisme de synchronisation utilisé pour contrôler l'accès à une ressource partagée dans un environnement multi-thread ou multi-processus. C'est comme un gardien qui permet à un certain nombre de personnes d'entrer dans une pièce, mais qui bloque l'accès lorsque la capacité maximale est atteinte.

Contrairement à un lock (ou Monitor) qui n'autorise qu'un seul thread à la fois (exclusivité totale), un sémaphore autorise l'accès à N threads simultanément.

Il est crucial de distinguer les deux classes disponibles dans l'espace de noms System.Threading :

Caractéristique	SemaphoreSlim (Recommandé)	Semaphore (Classique)
Portée	Local au processus (votre application).	Peut être global au système (inter-processus).
Poids	Léger (Lightweight). Utilise moins de ressources.	Lourd. C'est un "wrapper" autour d'un objet noyau Windows (Win32).
Async	Supporte <code>WaitAsync()</code> (parfait pour <code>async/await</code>).	Ne supporte pas l'async nativement.
Usage	99% des cas modernes.	Communication entre deux applications distinctes.

Cas d'utilisation typiques

- **Limitation de débit (Throttling)** : Empêcher votre API de spammer une API tierce qui impose des limites (ex: max 10 requêtes/seconde).
- **Gestion de pool de ressources** : Limiter le nombre de connexions ouvertes vers une base de données SQL ou un service externe.
- **Téléchargements parallèles** : Vous avez une liste de 100 fichiers à télécharger, mais vous ne voulez en télécharger que 5 à la fois pour ne pas saturer la bande passante.

Mutex:

Mutex est l'abréviation de "Mutual Exclusion" (Exclusion Mutuelle). C'est une primitive de synchronisation qui garantit qu'un seul thread à la fois peut accéder à une ressource ou exécuter un bloc de code.

Binaire de comptage:

Un binaire de comptage est un type de sémaphore qui utilise une variable entière pour compter le nombre de ressources disponibles. C'est comme un compteur qui indique combien de places sont encore disponibles dans une salle, et qui se décrémente chaque fois qu'une personne entre.

Exclusion mutuelle:

L'exclusion mutuelle est un principe de synchronisation qui garantit que lorsqu'un processus ou un thread accède à une ressource partagée, aucun autre processus ou thread ne peut y accéder en même temps.

Dijkstra:

Edsger W. Dijkstra était un informaticien néerlandais, célèbre pour ses contributions fondamentales à la science informatique, notamment dans les domaines de la programmation, des algorithmes et de la théorie des graphes. Il est notamment connu pour l'algorithme de Dijkstra, qui permet de trouver le chemin le plus court dans un graphe pondéré.

Concurrence vs Parallélisme:

Le Parallélisme consiste à gérer plusieurs tâches à la fois, tandis que la Concurrency consiste à exécuter plusieurs tâches à la fois.

Le Parallélisme (Architecture)

- **Concept** : Plusieurs tâches (A, B) sont découpées en parties (Ra, Rb) qui peuvent s'exécuter dans le désordre, mais pas forcément en même temps. Par exemple, $A \rightarrow Ra, B \rightarrow Rb$, puis Ra et Rb peuvent être alternés sur un même cœur.
- **Sur un CPU simple cœur** : Le système d'exploitation utilise le "Time Slicing". Il donne 10ms au Thread A, puis met le Thread A en pause pour donner 10ms au Thread B. Cela va si vite qu'on a l'illusion de la simultanéité.
- **But** : Maximiser l'utilisation du CPU (ne pas rester inactif quand on attend une réponse réseau, par exemple) et la réactivité.

La Concurrency (Exécution)

- **Concept** : Plusieurs tâches (Pa, Pb) s'exécutent réellement en même temps sur plusieurs cœurs ou machines. Par exemple, $Pa \text{ et } Pb \rightarrow Ra$, c'est-à-dire que les calculs sont faits simultanément.
- **Sur un CPU multi-cœurs** : Chaque cœur peut exécuter une tâche différente en parallèle.

- **But** : Aller plus vite (Performance pure) en divisant un gros problème de calcul en sous-problèmes traités par chaque cœur.

Priority Inversion:

L'inversion de priorité est un bug de planification (scheduling) où une tâche de haute priorité est forcée d'attendre indéfiniment qu'une tâche de basse priorité finisse son travail, souvent à cause de l'interférence d'une tâche de priorité moyenne.

C'est un paradoxe : le thread le plus important du système est bloqué par le moins important, violant ainsi les règles de priorité.

Communication inter-processus (IPC):

La communication inter-processus (inter-process communication, IPC, en anglais) regroupe un ensemble de mécanismes permettant à des processus concurrents de communiquer. Ces mécanismes peuvent être classés en trois catégories :

- les mécanismes permettant l'échange de données entre les processus ;
- les mécanismes permettant la synchronisation entre les processus (notamment pour gérer le principe de section critique) ;
- les mécanismes permettant l'échange de données et la synchronisation entre les processus.

CORBEILLE CHECK

Question: Thread - Compléter le code:

```
using System; using System.Collections.Generic;
using System.Linq; using System.Text;

namespace WS_THREAD
{
    class Program
    {
        static void Main(string[] args)
        {

            System.Threading.Thread t =
            new System.Threading.Thread(
            new System.Threading.ThreadStart(
            () =>
            {
                for (int i = 0; i < 10; i++)
                {
                    Console.WriteLine("Work...{0}", i);
                    System.Threading.Thread.Sleep(1000);
                }
            }
            );

            t.Start();
            Console.Read();

        }
    }
}
```

La classe Thread se trouve dans l'espace de noms System.Threading. Pour simplifier, un objet de la classe Thread symbolise une tâche.

Question: Pool Thread - compléter le code :

```
using System; using System.Collections.Generic;
using System.Linq; using System.Text; using System.Threading;

namespace WS_THREAD
{
    // Déclaration du délégué
    public delegate void DELG(object o);

    class Program
    {
        static void Main(string[] args)
        {
            System.Threading.Thread t1;

            DELG d = (o) =>
            {
                short i = 10;
                string msg = (string)o;

                while (i > 0)
                {
                    Console.WriteLine("Work - > {0}::{1}", msg, i);
                    System.Threading.Thread.Sleep(1000);
                    i--;
                }
            };

            t1 = new System.Threading.Thread(
                new ParameterizedThreadStart(d.Invoke)
            );

            System.Threading.ThreadPool.QueueUserWorkItem(
                (state) =>
                {
                    t1.Start("T1");
                }
            );

            Console.Read();
        }
    }
}
```

Question: Synchronisation - compléter le code :

```
using System; using System.Collections.Generic;
using System.Linq; using System.Text; using System.Threading;

namespace WS_THREAD
{
    class Program
    {
        private delegate void DELG(object state);

        private static int var = 0;
        private static object _lock;

        static void Main(string[] args)
        {
            _lock = new object();

            DELG d = (state) =>
            {
                lock (_lock)
                {
                    string name_thread = (string)state;
                    ++var;
                    Console.WriteLine("Thread -> {0} ({1})", var, name_thread);
                }
            };

            System.Threading.Thread t1 =
                new System.Threading.Thread(new ParameterizedThreadStart(d.Invoke));

            System.Threading.Thread t2 =
                new System.Threading.Thread(new ParameterizedThreadStart(d.Invoke));

            t1.Start("T1");
            t2.Start("T2");

            Console.Read();
        }
    }
}
```

Question: Synchronisation - compléter le code :

```
using System; using System.Collections.Generic;
using System.Linq; using System.Text; using System.Threading;

namespace WS_THREAD
{
    class Program
    {
        private delegate void DELG(object state);

        private static string simu_cnx_db = "Utilisation de la base de données";
        private static System.Threading.Semaphore semaphore;

        static void Main(string[] args)
        {
            semaphore = new System.Threading.Semaphore(2, 2);

            DELG d = (state) =>
            {
                semaphore.WaitOne();

                for (int i = 0; i < 3; i++)
                {
                    string name_thread = (string)state;

                    Console.WriteLine(
                        "Thread -> {0} -- Etat -> {1}",
                        name_thread,
                        simu_cnx_db.ToString()
                    );

                    System.Threading.Thread.Sleep(2000);
                }

                semaphore.Release();
            };

            System.Threading.Thread t1 =
                new System.Threading.Thread(new ParameterizedThreadStart(d.Invoke));

            System.Threading.Thread t2 =
                new System.Threading.Thread(new ParameterizedThreadStart(d.Invoke));

            System.Threading.Thread t3 =
                new System.Threading.Thread(new ParameterizedThreadStart(d.Invoke));

            t1.Start("T1");
            t2.Start("T2");
```



```
        t3.Start("T3");  
        Console.Read();  
    }  
}  
}
```

WORKSHOP

ANALYSE ET RÉOLUTION : BURNOUT

CONCLUSION

CAPITALISATION

AAV	Commentaire	Statut
Définir Thread, Multi-threading, Mécanismes d'IPC, Ressource critique / Section critique, inter-blocage	Les définitions de Thread, Multi-threading, IPC, Ressource critique, Section critique et inter-blocage sont fournies avec des exemples et explications claires.	Acquis
Citer, Décrire et expliquer les principaux problèmes liés à la section critique	Les problèmes comme l'interblocage, la famine, l'inversion de priorité sont identifiés, expliqués et illustrés avec des solutions proposées.	Acquis
Citer, Décrire et expliquer les principaux mécanismes d'IPC et les organiser en catégories	Les mécanismes d'IPC (sémaphores, mutex, etc.) sont présentés, catégorisés et expliqués avec des cas d'utilisation.	Acquis
Implémenter les mécanismes de synchronisation en C#	Des exemples de code C# illustrent l'utilisation de Thread, ThreadPool, lock, Semaphore, Mutex pour la synchronisation.	Acquis

RESSOURCES

-